

Representação de números negativos

Números sem sinal

A representação de números sem sinal em um computador aproveita todos os bits do número para representar quantidades: de 0 até $2^n - 1$ (2^n valores diferentes, onde n é o número de bits).

Por exemplo, um número de 8 bits pode armazenar números binários de 00000000 até 11111111 (de 0 a 255 em decimal). Isto representa a **magnitude** do número.

Exemplo:

$n = 8$ bits

Números sem sinal: $0 \leq X \leq 255$

Números positivos e negativos

A representação dos números positivos e negativos em um computador também permite representar quantidades em função do número de bits do número. Entretanto, precisam reservar um bit para a representação do sinal (+ ou -). Isto é feito em geral acrescentando ao número um outro bit, chamado bit de sinal.

Quando trabalhamos com binários com sinal, somente podemos representar números com a metade da magnitude de um binário sem sinal, pois o bit mais significativo é reservado para o sinal, por exemplo:

$n = 8$ bits

Binário sem sinal: $0 \leq X \leq 255$

Binário com sinal: $-128 \leq X \leq 127$

A forma mais utilizada de representar números binários positivos e negativos é o método chamado **complemento de 2**.

Complemento de 2

Sinal e magnitude

No método do complemento de 2 o bit mais a esquerda (MSB) representa o sinal:

- 0 indica número positivo;
- 1 indica número negativo.

Os $(n - 1)$ bits restantes representam a magnitude do número.

Binários positivos

Para os números binários positivos, o bit de sinal é 0 e os $n-1$ bits restantes representam a magnitude do número, que pode ser determinada de forma direta.

Exemplo:

$$00101010_2 = +42_{10}$$

Binários negativos

Para representar os números binários negativos é necessário calcular o complemento de 2 do número, em dois passos:

- Calcula-se o complemento de 1 do número (veja abaixo);
- Soma-se 1 ao complemento de 1.

Despreza-se o transporte no bit mais significativo (chamado de **carry externo**), caso exista.

Complemento de 1

O complemento de 1 de um binário é o simétrico dele, com todos os bits complementados, incluindo o bit de sinal.

Complemento de 1: Troque 0 por 1 e vice-versa.

Exemplos

Veja a forma de representar números decimais com sinal como números binário com sinal no método do complemento de 2, usando um total de 5 bits (incluindo o bit de sinal).

- **Número decimal +13**

É positivo, portanto é representado de forma direta:

$$13_{10} = 1101_2$$

Anexando o sinal o temos:

$$+13 = 01101$$

|

Bit de sinal

- **Número decimal -13**

É negativo, portanto sua magnitude deve ser representada na forma de complemento de 2:

$$13_{10} = 1101_2 \text{ (magnitude)}$$

Calculando o **complemento de 2**:

0010 (complemento de 1)

+ 1

0011 (complemento de 2)

Anexando o sinal o temos:

$$-13 = 10011$$

|

Bit de sinal

Extensão do sinal

Nos exemplos anteriores foi necessário usar um total de cinco bits para representar os números com sinal, um bit a mais que o necessário para representar a magnitude do número.

Normalmente os computadores usam registros para armazenar os números binários que são múltiplos de quatro bits, como 4, 8, 16, 32 ou 64. Por exemplo, em um sistema que representa números de 8 bits, o bit mais significativo (MSB) é o sinal e os outros sete são a magnitude.

Veja o caso dos números dos exemplos anteriores:

$$\begin{array}{r} +13 = 00001101 \\ | \\ \text{Bit de sinal} \end{array}$$

Como é positivo, basta acrescentar zeros a esquerda.

$$\begin{array}{r} -13 = 11110011 \\ | \quad | \\ | \quad \text{Bit de sinal no formato de cinco bits} \\ \text{Extensão do bit de sinal no formato de oito bits.} \end{array}$$

Como é negativo, acrescentamos uns a esquerda.

Negação

A negação é a operação de conversão de um número positivo em seu equivalente negativo, ou de um número negativo em seu equivalente positivo (inversão do sinal).

Para realizar a negação de um número basta calcular seu complemento de 2.

Exemplo de binário com sinal de oito bits

Positivo:

$$+42_{10} = 00101010_2 \text{ (1 bit de sinal + 7 bits magnitude)}$$

Negativo:

Passo 1: calcula-se o complemento 1 do número

00101010 (positivo)

11010101 (complemento 1)

Passo 2: soma-se 1 ao complemento 1

$$\begin{array}{r} 11010101 \\ + \quad 1 \\ \hline \end{array}$$

$$11010110_2 = -42_{10} \text{ (bit de sinal + complemento de 2)}$$

Negação: a negação de um número negativo será o seu simétrico positivo:

$$11010110_2 = -42_{10} \text{ (negativo)}$$

Passo 1: calcula-se o complemento 1 do número

00101001 (complemento 1)

Passo 2: soma-se 1 ao complemento 1

00101001

+ 1

00101010₂ = 42₁₀ (simétrico positivo)

Para converter um número binário negativo, na notação complemento de 2 para decimal, soma-se os pesos dos bits 1, sendo o bit de sinal (mais significativo) com peso negativo.

Exemplo:

-128 64 32 16 8 4 2 1

1 1 0 1 0 1 1 0 = -128 + 64 + 16 + 4 + 2 = 42

Aritmética com complemento de 2

Subtração.

A subtração pode ser implementada como soma do complemento de 2, ignorando eventual bit de estouro (*carry* externo). Isto permite que a adição e a subtração sejam efetuadas pelo mesmo circuito digital.

Exemplo:

Operação: + 4₁₀ - 3₁₀ (representados em binários de 4 bits)

4₁₀ = 0100₂

3₁₀ = 0011₂

-3₁₀ = 1100 + 1 = 1101₂ (complemento de 1 + 1 = complemento de 2)

Realizando a soma do positivo com o negativo:

0100

+1101

10001

|

Bit de **carry** externo desprezado

Exercícios:

1. Represente os números decimais com sinal como números binários com sinal no sistema de complemento de 2, usando um total de 8 bits (bit de sinal + 7 bits de magnitude):

- +3
- -2
- +8
- -8
- +56
- -100

2. Efetue a subtração dos seguintes pares de números binários positivos, usando complemento de 2. Converter o resultado para decimal, indicando se é positivo ou negativo:

(00101101) - (00010010) -> números de 8 bits: sinal + magnitude

(00010010) - (00101101) -> números de 8 bits: sinal + magnitude

(000010001011) - (000000110101) -> números de 12 bits: sinal + magnitude

(000101011101) - (000011100110) -> números de 12 bits: sinal + magnitude

Exercícios para entrega (por meio de atividade própria no ava):

Converta os números decimais em binário de 8 bits, com sinal, e realize as operações indicadas usando soma, usando complemento de 2 para representar os negativos. Converter o resultado para decimal, indicando se é positivo ou negativo:

55 - 77

- 43 - 61

- 15 - 28

Multiplicação.

1. Número de bits do produto:

O número de bits do produto será o dobro do número de bits do multiplicando e do multiplicador. Por exemplo, de multiplicando e multiplicador forem de 8 bits, o produto será de 16 bits. O MSB é sempre o bit de sinal.

2. Sinal do produto:

- Se os dois números forem positivos, poderão ser multiplicados diretamente e o resultado será positivo (bit de sinal 0);
- Se os dois números forem negativos, representados em complemento de 2, deve-se obter o complemento de 2 dos números para convertê-los em positivos e em seguida multiplicá-los. O produto será positivo (bit de sinal 0);
- Se um número for positivo e o outro negativo, o negativo deverá ser convertido em positivo, através do complemento de 2, para então multiplicar os números. Como o resultado deve ser negativo, o produto deverá ser convertido em negativo, através do complemento de 2 (bit de sinal 1).

Exercícios

Converta os números decimais em binário de 8 bits, com sinal, e realize as operações de multiplicação indicadas, explicitando se os produtos são positivos ou negativos:

(+7) * (-6)

0 1 1 1 => +7

* 0 1 1 0 => +6

1 1 1 ← Carry (vai 1)

0 0 0 0

0 1 1 1 0

0 1 1 1 0 0

+0 0 0 0 0 0 0

0 1 0 1 0 1 0 => +42

Deve-se passar o resultado para negativo, pois o produto de (+7) * (-6) é negativo.

0 0 1 0 1 0 1 0 => 32 + 8 + 2 = +42

Complemento de 1: 1 1 0 1 0 1 0 1

+ 1

Complemento de 2: 1 1 0 1 0 1 1 0 => -128 + 64 + 16 + 4 + 2 = -42

Exercícios para entrega (por meio de atividade própria no ava)

Converta os números decimais em binário de 8 bits, com sinal, e realize as operações de multiplicação indicadas, explicitando se os produtos são positivos ou negativos:

a) $(+10) * (-9)$

b) $(-7) * (-5)$

Carry e Overflow

Carry interno

Em uma soma (ou subtração em complemento de 2) de números binários sem sinal, toda vez que "vai 1" em uma coluna para a coluna da esquerda, temos um *carry* interno.

Exemplo

```
  1  <- vai 1 (carry interno)
101010
+ 010011
-----
111101
```

Carry externo

Caso o "vai 1" ocorra no bit mais significativo (MSB) temos um *carry* externo. Neste caso, este bit de carry poderá ser utilizado para indicar que o resultado da soma não cabe nesta quantidade de bits.

Exemplo

```
 1  1  <- vai 1 (O "vai 1" do bit MSB é um carry externo, o outro um carry interno)
101010 (6 bits)
+ 110011 (6 bits)
-----
1011101 (7 bits!)
```

Overflow aritmético

O *overflow* ocorre quando o resultado de uma operação supera a capacidade do registro usado para guardar este resultado.

Para somas ou subtração de números com sinal (+ ou -), o *overflow* ocorre caso o sinal do resultado não seja aquele que seria o esperado (por exemplo, um resultado negativo da soma de dois números positivos):

Exemplo

Soma dos números binários +9 com +8, ambos com 4 bits de magnitude e 1 bit de sinal:

```
+9 -> 0 1001
+8 -> 0 1000
-----
1 0001 <- Magnitude incorreta
|
Sinal incorreto
```

Explicação do *overflow* no exemplo dado:

Com números em complemento de 2 é possível representar a faixa de valores entre $-2^{n-1} \leq X \leq 2^{n-1} - 1$, onde n é o número de bits. No exemplo $n = 5$, logo pode representar valores entre $-2^{5-1} \leq X \leq 2^{5-1} - 1$, ou seja, $-16 \leq X \leq 15$. Como o resultado da operação $(+9) + (+8)$ teria que dar 17, ocorreu o *overflow*.